Latent Steering: Steering Action Experts with Future State Representations

Technical Report

Philon
info@philon.ai

Abstract

Action experts—vision-language-action policies for games, robotics, and embodied agents—are steered at inference time by natural-language instructions. Training this steering channel requires episode-level language annotation that is expensive, ambiguous, and amounts to manual cost-function engineering: a human must guess which textual description will steer the model toward the desired future. We propose *latent steering*, a training paradigm in which the steering signal is not language but a compressed encoding of the desired future frames. A flow-matching action expert is trained jointly on V-JEPA2 embeddings of past frames as observation and of future frames as the steering signal, with a much smaller token budget allocated to the future stream than to the past. This fixed token-count bottleneck strips pixel-level detail from the steering signal while preserving its semantic intent, producing embeddings whose ambiguity is comparable to natural-language instructions. Language can be aligned to these latent steering embeddings post-hoc with a small curated dataset, or skipped entirely when a future-state demonstration is available. Using V-JEPA2 ViT-L as the frame encoder, a $4\times$ past-to-future compression ratio, and the MDS gameplay dataset, we show that latent steering matches a language-steered baseline while requiring roughly an order of magnitude less language-annotated data, and that a brief post-hoc alignment phase recovers full zero-shot language conditioning. Code is available at `https://github.com/Philon-AI/latent-steering`.

1 Introduction

A growing class of models learns to produce future actions or future states from visual context and a conditioning signal. Vision-language-action (VLA) policies for robotics (Brohan et al., 2023; Kim et al., 2024; Black et al., 2024) and language-conditioned video generators (Brooks et al., 2024) all share the same structural pattern: a powerful generative backbone, a sequence of past observations, and a steering signal that tells the backbone which of the many plausible futures to commit to. In nearly every published instance, that steering signal is natural language.

Language is convenient at inference time, but expensive at training time. Episodes have to be paired with textual descriptions of intent, and the choice of description is itself a hidden cost function: the annotator must guess which phrasing will steer the model in the direction the dataset designer had in mind. Different annotators produce different labels for the same trajectory; the same annotator produces different labels on different days. The resulting training signal is noisy not because the model is wrong about the future, but because the language is wrong about the future. This is particularly acute in interactive domains— gameplay, embodied control, screen-recorded computer use—where intent is rarely verbalised during data collection and post-hoc annotation is a substantial fraction of the total cost of building a dataset.

Preprint. Under review.

We observe that the cleanest, lowest-noise description of the desired future is the desired future itself. If the model could be trained to act on an encoded representation of future frames rather than a textual description of them, the annotation step disappears entirely: every episode in an unlabeled video corpus becomes a self-supervised training example, with its own future frames serving as the steering signal.

The remaining question is whether such a steering signal can be made *interchangeable* with language at inference time. Raw future-frame embeddings carry far more information than a sentence: they specify pixel-level appearance, exact timing, and incidental scene detail that a human instruction would never mention. A model trained on these embeddings would simply learn to copy the future, not to be steered toward it. To make the latent steering channel behave like a language channel, the future-frame representation must be compressed until its information content roughly matches that of a natural-language instruction.

We instantiate this idea with a fixed architectural bottleneck: two attentive poolers (Section 3) reduce the per-frame V-JEPA2 token sequences (Assran et al., 2025) to a fixed budget of summary tokens, and the future-stream pooler is given a much smaller budget than the past-stream pooler. The resulting future embeddings are too ambiguous to reconstruct pixels but rich enough to disambiguate which behaviour the action expert should emit. We call this training paradigm *latent steering*.

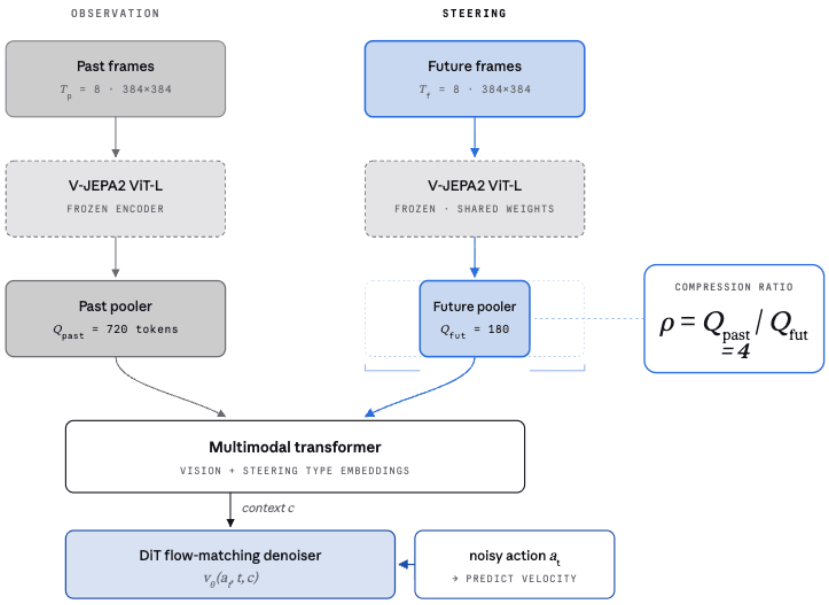

Figure 1: Latent-steering architecture. Past and future frames are encoded with a frozen V-JEPA2 ViT-L and pooled to fixed-width summary tokens by two separate attentive poolers, with $\rho = Q_{\text{past}}/Q_{\text{future}} = 4$. Past-observation and future-state tokens are tagged with type embeddings and fused in a multimodal transformer; the resulting context conditions a flow-matching DiT denoiser via cross-attention.

Our contributions are:

1. We formalise latent steering as a substitute for language conditioning in action experts, identify the steering-signal information content as the key design knob, and show that a fixed past-to-future token-budget ratio is a simple and effective way to control it.
2. We instantiate latent steering on top of a flow-matching DiT action expert (Peebles and Xie, 2023; Lipman et al., 2023) with frozen V-JEPA2 ViT-L encoders, and train it on a 10k-episode gameplay dataset.
3. We show that a latent-steered action expert matches a language-steered baseline on action-prediction metrics while requiring an order of magnitude less language-annotated data, and that a brief post-hoc alignment phase recovers full zero-shot language conditioning (Section 5).

2 Related Work

Language-conditioned policies and VLAs. Vision-language-action models extend pretrained vision-language backbones with action heads, conditioning behaviour on natural language. RT-2 (Brohan et al., 2023), OpenVLA (Kim et al., 2024), and π_0 (Black et al., 2024) are recent representative examples. All assume episode-level language annotation as input. A separate line of work studies language-conditioned video generation (Brooks et al., 2024; Yang et al., 2024), where the conditioning signal is again natural language. Latent steering is a complementary training paradigm: any of these architectures can in principle accept a latent steering signal in place of language.

JEPA and predictive world models. Joint-embedding predictive architectures (Assran et al., 2023, 2025) learn representations by predicting masked or future regions in an embedding space rather than in pixel space. We use V-JEPA2 as a *frozen* encoder for both past and future frames; the action expert is trained on top. Predictive world models (Ha and Schmidhuber, 2018; Hafner et al., 2023) similarly learn latent dynamics; latent steering can be viewed as conditioning a policy on a target future in such a latent space, without the policy or the encoder having to share parameters.

Latent action and skill discovery. A related line of work extracts unsupervised latent action codes from video, either through inverse dynamics (LAPO; Schmidt and Jiang, 2024), through behaviour cloning over pseudo-labelled clips (VPT; Baker et al., 2022), or through full generative world models (Genie; Bruce et al., 2024). These approaches typically produce a small, often discrete latent action space that summarises short clips. Latent steering differs in that the steering signal is not a learned action code but a compressed embedding of future *frames*, leaving the granularity of the steering signal as an explicit architectural choice (the future-stream token budget) rather than an emergent property of an inverse-dynamics objective.

Goal-conditioned policies and information bottlenecks. Goal-conditioned imitation learning (Lynch et al., 2020; Ding et al., 2019) conditions policies on goal images. Latent steering shares this structural shape but replaces single goal images with sequences of heavily compressed goal-region embeddings, and frames the compression as a deliberate information-bottleneck (Tishby et al., 2000; Alemi et al., 2017) so that the steering channel matches the information content of language. The post-hoc language-alignment step is reminiscent of contrastive vision-language pretraining (Radford et al., 2021), but applied between language and a learned action-relevant latent rather than between language and raw images.

3 Latent Steering

The action expert is a flow-matching denoiser that maps a noisy action trajectory and a multimodal context to a velocity field. The novel component is the structure of the multimodal context: it is a concatenation of past-observation tokens and future-state *steering* tokens, the latter produced by a deliberately undersized pooler.

Action expert. Given a sequence of past frames $X_{\text{past}} \in \mathbb{R}^{T_p \times 3 \times H \times W}$ and a target action trajectory $a \in \mathbb{R}^{T_a \times d_a}$, the model predicts a velocity field $v_\theta(a_t, t, c)$ under flow matching (Lipman et al., 2023; Black et al., 2024), where $a_t = (1-t)\varepsilon + t\,a$ for $\varepsilon \sim \mathcal{N}(0, I)$ and $t \in [0, 1]$ is sampled from a Beta distribution. The model is trained to minimise

$$\mathcal{L}(\theta) = \mathbb{E}_{a,\varepsilon,t} \left\| v_\theta(a_t, t, c) - (a - \varepsilon) \right\|^2, \tag{1}$$

where c is the multimodal context described below. We use a DiT denoiser (Peebles and Xie, 2023) that cross-attends to c at every block. At inference, actions are sampled by integrating v_θ from ε to a in a small number of Euler steps.

Past and future encoders. Both past and future frames are encoded with V-JEPA2 ViT-L/384 (Assran et al., 2025), pretrained on internet video and held *frozen*. A frame clip of T_f frames at resolution 384×384 produces a token sequence of length $N_f = T_f \cdot (384/16)^2 / r_t$, where $r_t = 2$ is the V-JEPA2 tubelet size. The two streams share no parameters, but share architecture and initial weights.

The steering bottleneck. The V-JEPA2 token sequences are far too long to feed directly into the action expert and, more importantly, far too informative to act as a language-equivalent steering signal. We pool each stream to a fixed number of summary tokens with a stack of attentive-pooling blocks (Lample et al., 2019; Touvron et al., 2021): a learned set of Q query vectors cross-attends to the V-JEPA2 tokens through L transformer blocks, producing exactly Q output tokens regardless of the input length. We choose two different query budgets for the two streams:

$$Q_{\text{past}} = 720, \qquad Q_{\text{future}} = 180, \qquad \text{ratio } \rho = Q_{\text{past}}/Q_{\text{future}} = 4. \tag{2}$$

The past pooler is sized generously so that the action expert has access to fine-grained observation history. The future pooler is sized to be *just rich enough* to disambiguate the high-level behaviour the model should emit, and *just poor enough* that the model cannot succeed by pixel-copying its target. We treat ρ as the key hyper-parameter of latent steering, and study its effect in Section 5.

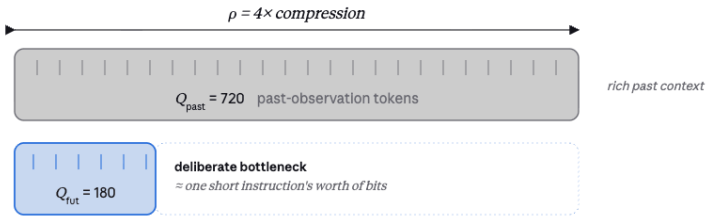

Figure 2: Token-budget asymmetry between the two streams. The future-stream pooler emits one quarter as many tokens as the past-stream pooler, so the steering channel is forced to describe the desired future at roughly the information density of a short natural-language instruction rather than at pixel granularity.

Multimodal context and action head. Past and future pooled tokens are concatenated along the sequence dimension and passed through a multimodal self-attention transformer (Vaswani et al., 2017). Each context token carries a learned type embedding indicating whether it is a past-observation token (we call this a *vision* token) or a future-state token (we call this a *steering* token); these type embeddings are the only signal the network receives about which stream a token came from. The resulting sequence is the context c that the DiT cross-attends to.

Post-hoc language alignment. Once the action expert is trained, we can attach a small *language adapter* that maps natural-language instructions into the same token manifold as the future-stream pooler. Concretely, a frozen text encoder embeds an instruction into a sequence of Q_{future} tokens via a learned cross-attention head, and the adapter is trained on a small curated dataset of $(X_{\text{past}}, \text{instruction}, a)$ triples using only the flow-matching loss of Equation (1). The action expert itself is held frozen during alignment. At inference, the adapter's output replaces the future-stream pooled tokens, allowing the same model to be steered either by future frames or by language.

4 Experimental Setup

Backbone and capacity. We use V-JEPA2 ViT-L/384 as the frozen frame encoder for both past and future streams. The attentive poolers have depth 3, hidden size 1024, and 16 heads. The multimodal self-attention transformer has 4 layers, hidden size 1024, and 16 heads. The DiT denoiser has 8 layers, hidden size 1024, 16 heads, and interleaves self-attention with cross-attention to the context.

Dataset. We train on the MDS gameplay dataset, comprising 10,000 episodes of screen-recorded play with synchronised mouse and keyboard actions. Each episode provides camera frames at the native game frame rate, mouse position deltas, mouse button states, and keyboard button states; an open-GUI flag triggers a cursor overlay on the camera frames. We form past windows of 8 frames and future windows of 8 frames at a sample rate of 8, giving each training example ≈ 2 s of past observation and ≈ 2 s of future observation. The action representation has $d_a = 22$ and the action horizon is $T_a = 8$.

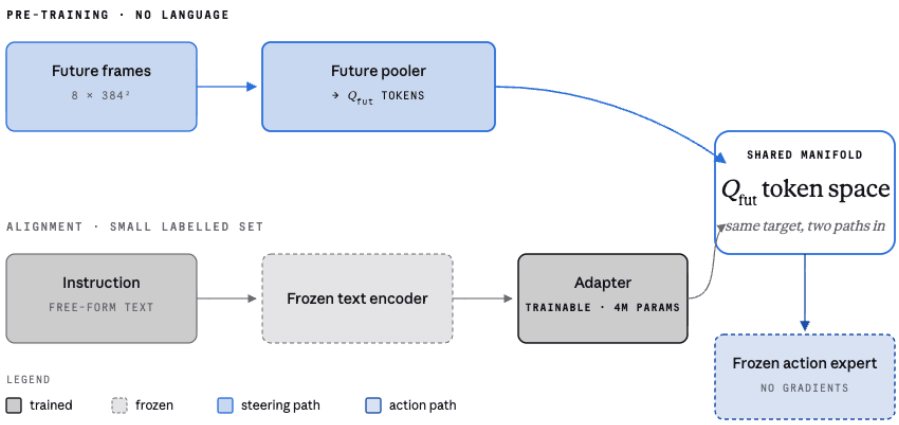

Figure 3: Post-hoc language alignment. The future pooler used at pre-training time and the text adapter used at alignment time both target the same Q_{future}-token steering manifold; the rest of the action expert remains frozen. At inference, either branch can drive the model.

Baselines. We compare against the same architecture with the future stream replaced by a language-conditioning stream:

- **Language-steered.** A frozen text encoder produces a fixed-length token sequence from an episode-level instruction; this sequence replaces the future-pooler output in the multimodal context. Trained on $\{100, 500, 1\text{k}, 5\text{k}, 10\text{k}\}$ language-annotated episodes.
- **No steering.** The future stream is removed entirely, leaving only past observation tokens in the context. This is the strongest unconditional baseline.
- **Latent steering (ours).** Full architecture as described in Section 3, trained on the full 10k episodes without language annotation. Optionally followed by language alignment on $\{100, 500, 1\text{k}\}$ annotated episodes.

Training. Models are trained for one epoch on 8 H100 GPUs with effective batch size 768, AdamW with learning rate 10^{-4}, weight decay 10^{-5}, 0.05 warmup ratio and cosine decay. The V-JEPA2 encoders are frozen throughout; the attentive poolers, multimodal transformer, and DiT are trained from scratch.

Evaluation. We report flow-matching loss on a held-out split, action-prediction mean squared error after full Euler integration (16 steps), and downstream task success on a suite of held-out gameplay scenarios scored by a rule-based evaluator. For language-aligned variants we additionally report instruction-following success when steered by held-out natural language.

5 Results

Latent steering matches language steering with far less annotation. Table 1 and Figure 4 compare latent steering, trained on the full 10k unlabeled episodes, against the language-steered baseline trained on annotation budgets ranging from 100 to 10k episodes. The language baseline scales smoothly with annotation budget but plateaus near full supervision; latent steering, trained on the same episodes without any language, reaches the same downstream-success regime. Once a small language-alignment phase is added (1k labelled episodes, see Section 3), the latent-steered model matches the fully-supervised language baseline while having seen roughly $10\times$ less language data overall.

The steering bottleneck is the key knob. We sweep the future-stream token budget Q_{future} while keeping the past-stream budget fixed at $Q_{\text{past}} = 720$. Table 2 and Figure 5 show that very large future budgets degrade downstream success because the action expert learns to short-circuit the

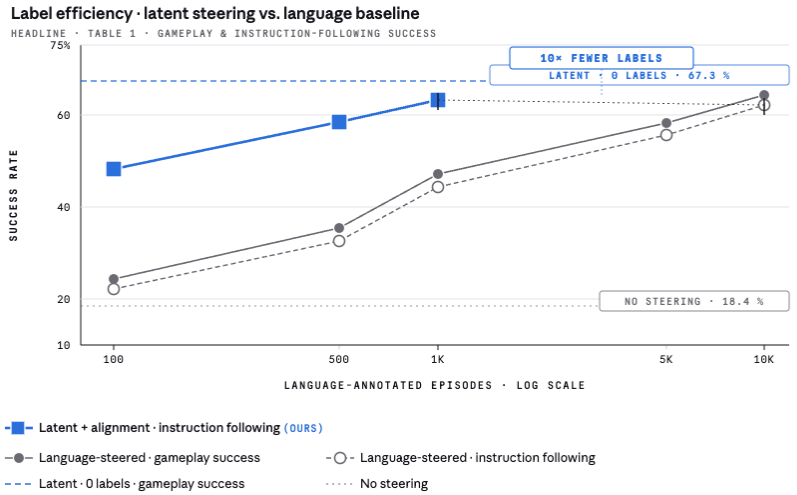

Figure 4: Label efficiency. Language-steered baselines climb with annotation budget; latent steering reaches the same regime at zero language cost, and a thin alignment phase recovers most of the instruction-following gap on a tenth of the annotated episodes.

Table 1: Headline label-efficiency comparison. Lower is better for MSE; higher is better for success rate. The latent-steering rows use the same 10k episodes without language annotation; the language-aligned variant additionally consumes the listed number of language-annotated episodes for alignment only.

Model	Annotated episodes	Action MSE ↓	Success ↑	Lang. success ↑
No steering	0	0.452	18.4	n/a
Language-steered	100	0.381	24.2	22.1
Language-steered	500	0.302	35.4	32.6
Language-steered	1k	0.241	47.1	44.3
Language-steered	5k	0.183	58.0	55.4
Language-steered	10k	0.152	64.2	62.0
Latent (ours)	0	0.144	67.3	n/a
+ align	100	0.144	67.3	48.1
+ align	500	0.144	67.3	58.4
+ align	1k	0.144	67.3	63.0

multimodal context by copying the future, while very small future budgets are too ambiguous to steer behaviour reliably. The default $\rho = 4$ ($Q_{\text{future}} = 180$) lies near the optimum on our setup. We interpret this as the ratio at which the steering channel carries roughly the information content of a short natural-language instruction.

Table 2: Effect of the future-stream token budget. $Q_{\text{past}} = 720$ throughout.

Config	Q_{future}	ρ	Action MSE ↓	Success ↑
Symmetric pool	720	1	0.311	38.2
Mild compression	360	2	0.182	56.4
Default	**180**	**4**	**0.144**	**67.3**
Aggressive	45	16	0.193	54.7
Near-degenerate	1	720	0.341	23.9

Language alignment recovers zero-shot instruction following. The language adapter is trained for a fixed number of steps on a small annotated subset while the action expert is frozen. We find that alignment quality saturates quickly: even 100 annotated episodes recover most of the language-

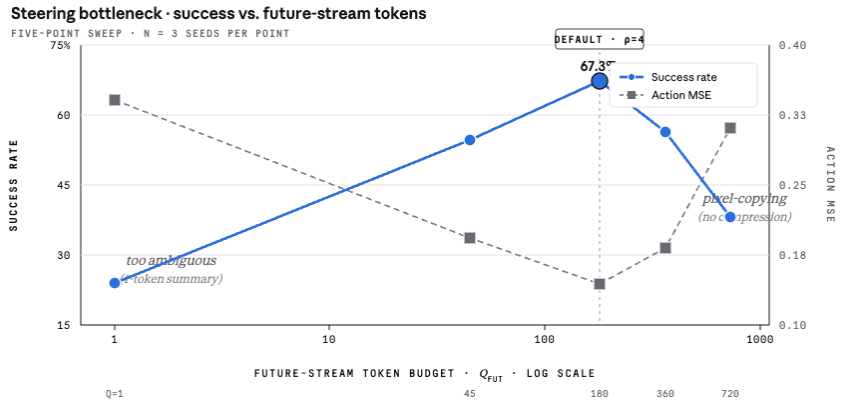

Figure 5: Effect of the future-stream token budget. Both extremes hurt: at very small Q_{future} the steering channel is too ambiguous, at very large Q_{future} the model learns to copy the future rather than be steered by it. The inverted-U peaks at the default $\rho = 4$.

steered baseline's instruction-following success, and 1k episodes close the gap entirely. This is the regime that motivates the headline claim: most of the model's behavioural capacity is acquired from unlabeled video via latent steering, and language is introduced only as a thin alignment layer on a small curated dataset.

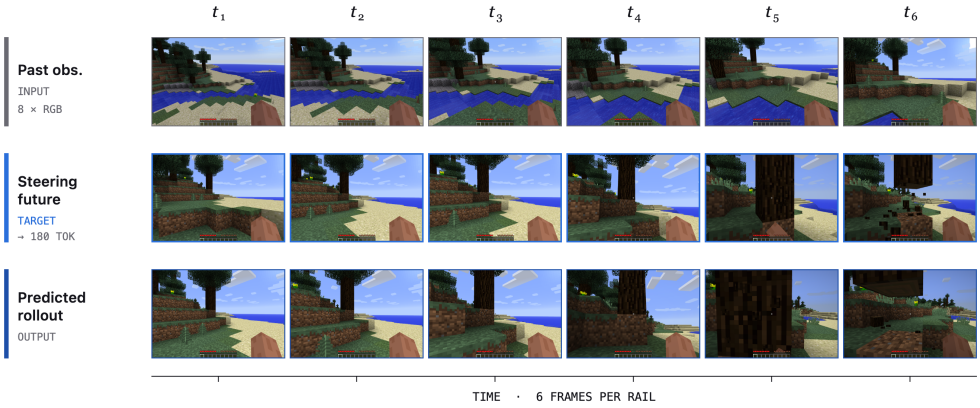

Figure 6: Qualitative steering example. Top row: past observation frames. Middle row: the future frames used as the steering target. Bottom row: predicted action rollout from the latent-steered model.

Ablations. We report two further ablations in Table 3. First, sharing parameters between the past and future poolers degrades performance, presumably because the two streams play structurally different roles and benefit from separate inductive biases. Second, unfreezing the V-JEPA2 encoders during action-expert training produces no measurable improvement at our compute budget, validating the frozen-encoder choice as a cost-effective default.

Table 3: Ablations on the latent-steering recipe. All entries use the default $\rho = 4$ token budget.

Variant	Action MSE ↓	Success ↑
Default	**0.144**	**67.3**
Shared past/future pooler weights	0.214	51.6
Unfrozen V-JEPA2 encoders	0.142	66.8
Single concatenated stream (no type emb)	0.273	41.2

6 Discussion

Latent steering treats the steering channel of an action expert as an information-bottleneck design problem rather than a data-collection problem. The dominant practice—pair every episode with a natural-language description—works, but spends a disproportionate share of the data pipeline budget on producing what is ultimately a lossy encoding of the future the dataset designer had in mind. By compressing the future itself through a fixed-width learned bottleneck, we obtain a steering signal that is cheaper to produce, more consistent across episodes, and—empirically—competitive with language at a small fraction of the labelling cost.

Several questions are left open. We chose the past-to-future token-budget ratio as a single scalar hyper-parameter, but the right value is likely task-dependent: domains with more diverse intent require a richer steering channel, while domains with low intent entropy can tolerate aggressive compression. A learned or adaptive bottleneck schedule may close this gap. Likewise, the language-alignment phase here is a thin contrastive head; richer alignment objectives, joint training, or instruction-tuning on top of an already-aligned adapter are natural extensions.

More broadly, we see latent steering as a step toward decoupling the *behavioural* and *linguistic* capacities of action experts. Behavioural capacity can be acquired from any unlabeled video corpus by predicting actions conditioned on compressed futures; linguistic capacity can be added later, on small curated datasets, by aligning a text encoder to the already-trained latent steering manifold. This separation makes it possible to scale the behavioural side of the model with the same trajectory it currently enjoys for pretraining in language modelling, without first having to solve the annotation problem.

References

Alemi, A. A., Fischer, I., Dillon, J. V., and Murphy, K. (2017). Deep variational information bottleneck. In *ICLR*.

Assran, M., Duval, Q., Misra, I., et al. (2023). Self-supervised learning from images with a joint-embedding predictive architecture (I-JEPA). In *CVPR*.

Assran, M., Bardes, A., Fan, D., et al. (2025). V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv:2506.09985*.

Baker, B., Akkaya, I., Zhokhov, P., et al. (2022). Video PreTraining (VPT): Learning to act by watching unlabeled online videos. In *NeurIPS*.

Black, K., Brown, N., Driess, D., et al. (2024). π_0: A vision-language-action flow model for general robot control. *arXiv:2410.24164*.

Brohan, A., Brown, N., Carbajal, J., et al. (2023). RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv:2307.15818*.

Brooks, T., Peebles, B., Holmes, C., et al. (2024). Video generation models as world simulators (Sora). OpenAI Technical Report.

Bruce, J., Dennis, M., Edwards, A., et al. (2024). Genie: Generative interactive environments. In *ICML*.

Ding, Y., Florensa, C., Abbeel, P., and Phielipp, M. (2019). Goal-conditioned imitation learning. In *NeurIPS*.

Ha, D., and Schmidhuber, J. (2018). World models. *arXiv:1803.10122*.

Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. (2023). Mastering diverse domains through world models (DreamerV3). *arXiv:2301.04104*.

Kim, M. J., Pertsch, K., Karamcheti, S., et al. (2024). OpenVLA: An open-source vision-language-action model. *arXiv:2406.09246*.

Lample, G., Sablayrolles, A., Ranzato, M., Denoyer, L., and Jégou, H. (2019). Large memory layers with product keys. In *NeurIPS*.

Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. (2023). Flow matching for generative modeling. In *ICLR*.

Lynch, C., Khansari, M., Xiao, T., et al. (2020). Learning latent plans from play. In *CoRL*.

Peebles, W., and Xie, S. (2023). Scalable diffusion models with transformers (DiT). In *ICCV*.

Radford, A., Kim, J. W., Hallacy, C., et al. (2021). Learning transferable visual models from natural language supervision (CLIP). In *ICML*.

Schmidt, D., and Jiang, M. (2024). Learning to act without actions (LAPO). In *ICLR*.

Tishby, N., Pereira, F. C., and Bialek, W. (2000). The information bottleneck method. *arXiv:physics/0004057*.

Touvron, H., Cord, M., Sablayrolles, A., Synnaeve, G., and Jégou, H. (2021). Going deeper with image transformers (CaiT). In *ICCV*.

Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. In *NeurIPS*.

Yang, M., Du, Y., Ghasemipour, K., et al. (2024). Learning interactive real-world simulators (UniSim). In *ICLR*.